

CSC3210 Computer Graphics — Fall 2026

Unit 5 – Colors, Rasterization, & Textures

5.1 Basic Color Theory

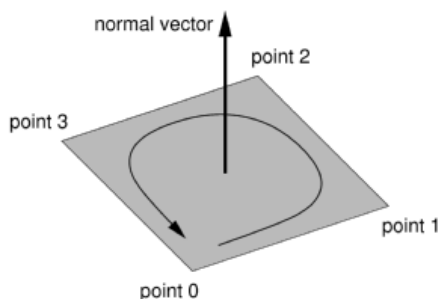
- Photons!
 - Wavelength
 - Visible Spectrum (390 nm to 720 nm)
- Color spaces
 - RGBA (red-green-blue-alpha) color space, the one we will use and the one OpenGL uses
 - HSV (hue-saturation-value)
 - CMYK (cyan-magenta-yellow-black)
- Colors are represented by 4 8-bit values (how many unique colors are there?)
- Read:
<https://colormatters.com/color-and-design/basic-color-theory>
- Hexidecimal color encodings:
<https://marketing.istockphoto.com/blog/hex-colors-guide/>

5.2 Color Stuffs!

- The fragment shader is responsible for coloring **fragments**
 - A Fragment is a collection of values produced by the **Rasterizer**.
 - Each fragment represents a sample-sized segment of a rasterized **Primitive**.
 - The size covered by a fragment is related to the pixel area, but rasterization can produce multiple fragments from the same triangle per-pixel, depending on various multisampling parameters and OpenGL state. There will be at least one fragment produced for every pixel area covered by the primitive being rasterized.
 - The **Fragment Shader** is executed once per fragment.
- This can be provided in a variety of ways
 - Using the fragment shader to specify a color (hard-coded into the code)
 - Color data can be provided for each input vertex and passed to the fragment shader
 - Data could be provided to the fragment shader and used in a computation to generate a color (such as light)
 - Other data, like an image, can be referenced by the fragment shader

5.2.1 Culling

- Only primitives that *face the camera* are drawn. Anything else is “culled” (ignored).
- To determine the *front face* of a primitive, use the **right-hand rule** - curl your right hand fingers (this is counter-clockwise), and whichever way your thumb is facing is the “front” face of the geometry



- If the camera can see the normal vector protruding from the face, then it is drawn and everything else is culled.
- In `Three.js` you can control which face is drawn and which is culled (if any) using the material's "side" property - see the "Right-Hand Rule Example"

5.2.2 Rasterization

- Determine which screen locations (pixels) are covered by each piece of geometry
- **Gouraud shading** - Linear interpolation of vertex and color data to compute the relevant data for other pixels besides the specifically designed vertices that are affected
- This is sent as input to the fragment shader

5.3 Operating on Fragments

After rasterization of a primitive, fragments go through a series of processing steps before they are sent to the fragment shader for the corresponding pixels (determined during rasterization)

1. Scissor test
2. Multisample fragment operations
3. Stencil test
4. Depth test
5. Blending
6. Dithering
7. Logical operations

The tests can be enabled or disabled at will.

5.3.1 Scissor Test

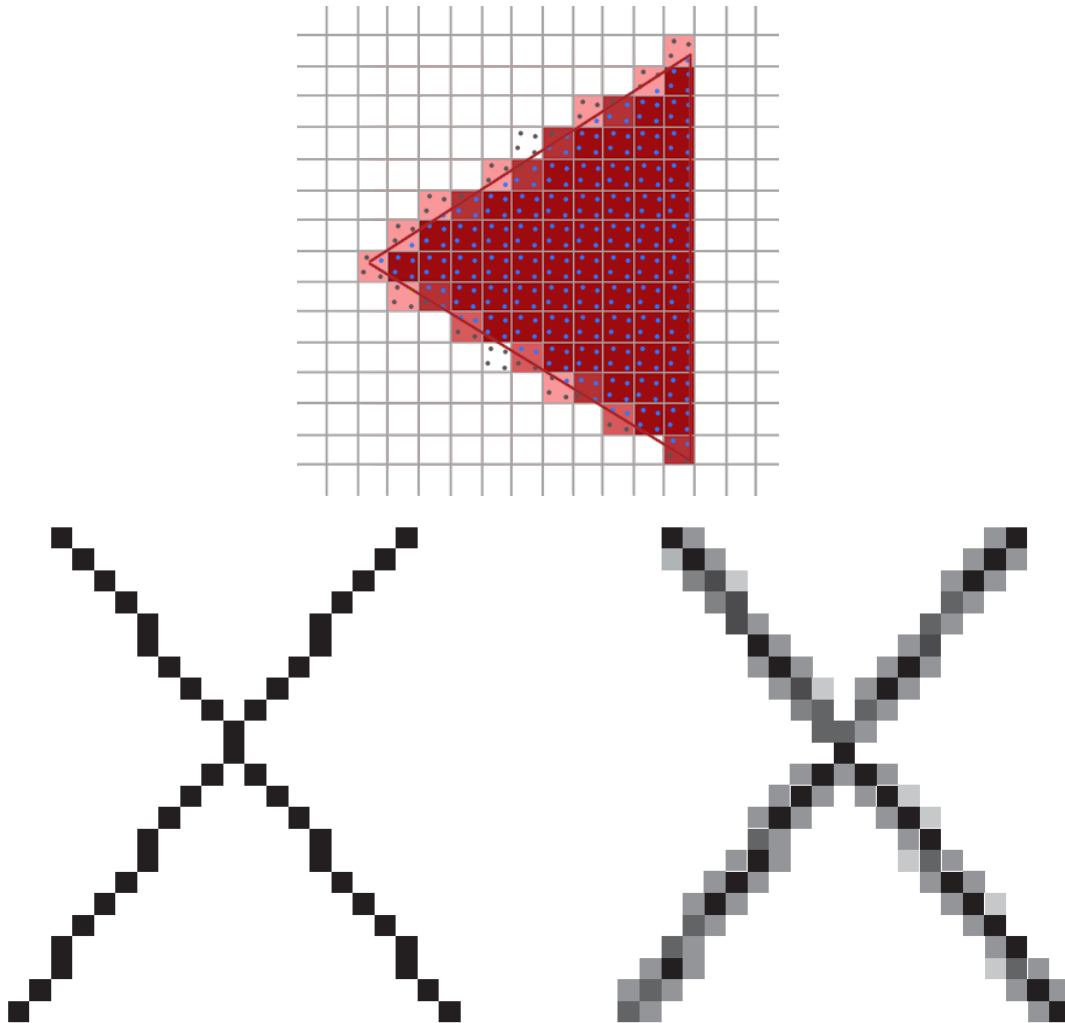
- The *scissor box* is a rectangular portion of the drawing window and restricts all drawing to its region
- After a scissor area (x,y,w,h) has been specified (in window coordinates), and the scissor test enabled, all subsequent rendering calls (including clearing) affects only the area within the scissor area.
- This is useful for producing simple "stenciled" animation.
- You can specify the scissor box with

```
renderer.setScissor(x, y, width, height);
```

- where `setScissor` is called on an object of type `WebGLRenderer`
<https://threejs.org/docs/index.html#api/en/renderers/WebGLRenderer>
 - `x` and `y` refer to the lower-left corner of the box
 - `width` and `height` are as you would expect
- If the test is enabled, all rendering (including clearing) is restricted to the scissor box
 - `renderer.getScissorTest( )` to test if it's enabled
 - `renderer.setScissorTest( true )` to enable it (or `false` to disable it)

5.3.2 Multisample Fragment Operations

Each pixel is *super-sampled*, meaning that it is broken up into four quadrants (can be sampled at higher densities for more accurate, yet computationally expensive result). This is called **anti-aliasing** as well.



- Calculates *fragment coverage values* (essentially, percentages of the pixels that are covered)
- Ignores alpha values unless told otherwise
- The “strength” of the color is determined by the number of quadrants that the geometry occupies (see image)
 - Note the color differences between pixels with 0, 1, 2, 3, or 4 quadrants occupied (dots within the geometry).
 - This “blurring” effect produces anti-aliasing
- Enabled/disabled using the initialization of the renderer:

```
renderer = new THREE.WebGLRenderer({ antialias: true });
```

5.3.3 Stencil Test

- Only takes place if there is a stencil buffer, which needs to be requested when the window is created
- Simply compares some reference value against the value stored in the stencil buffer at a pixel
 - Depending on the result of the test, the value in the stencil buffer can be changed

```
var renderer = new THREE.WebGLRenderer({stencil: true}); //  
    Also the default  
// OR  
var context = renderer.getContext();  
context.enable(context.STENCIL_TEST);
```

* Function values can be:

- NEVER
- ALWAYS
- LESS
- LEQUAL
- EQUAL
- GEQUAL
- GREATER
- NOTEQUAL

* For instance, if using LESS, then the fragment passes if **ref** is less than the value in the stencil buffer

```
// stencilFunc(func, ref, mask);  
// ref - reference value for the stencil test, the default  
// is 0  
// mask - a bit-wise mask that is used to AND the  
// reference value and the stored stencil value when  
// the test is done, the default value is all 1  
context.stencilFunc(context.EQUAL, 1, 0xffffffff);
```

– You can specify how the data in the stencil buffer is modified when the fragment passes or fails

```
// header: void gl.stencilOp(fail, zfail, zpass);  
context.stencilOp(context.KEEP, context.KEEP, context.KEEP);
```

– **fail** (what to do when the test fails), **zfail** (what to do when the test passes but the depth test fails, see next section), and **zpass** (what to do when both tests pass) are functions that could be

- * KEEP
- * ZERO
- * REPLACE
- * INCR
- * INCR_WRAP
- * DECR
- * DECR_WRAP
- * INVERT

– They default to KEEP

5.3.4 Depth Test

- If the incoming object's associated pixel is closer to the camera than the object occupying that pixel in the depth buffer, it is overwritten (the *depth test*)

```
// 'true' is the default value
var renderer = new THREE.WebGLRenderer({depth: true});
// OR
var context = renderer.getContext();
// Sets the depth test function to ALWAYS
context.depthFunc( context.ALWAYS );
```

- `glDepthTest(GLenum func)` uses the same function constants as `glStencilFunc` above
 - Passes based on the comparison test sent to it
 - Can be used to change which pixels are considered “closer”
 - The default is, of course, LESS

```
context.depthFunc( context.EQUAL );
```

5.3.5 Polygon Offset

- *Slightly* move object away from the camera or toward the camera to prevent anti-aliasing artifacts if you draw two objects on top of one another at the same z-value (called *z-fighting*)
- In order to determine how much the objects need to be offset, OpenGL needs to calculate the *depth slope*
- Depth slope is calculated in the x- and y- direction (change in depth over change in axis)
- The closer to 0 the depth slope (parallel to the clipping planes), the less of a depth adjustment is necessary

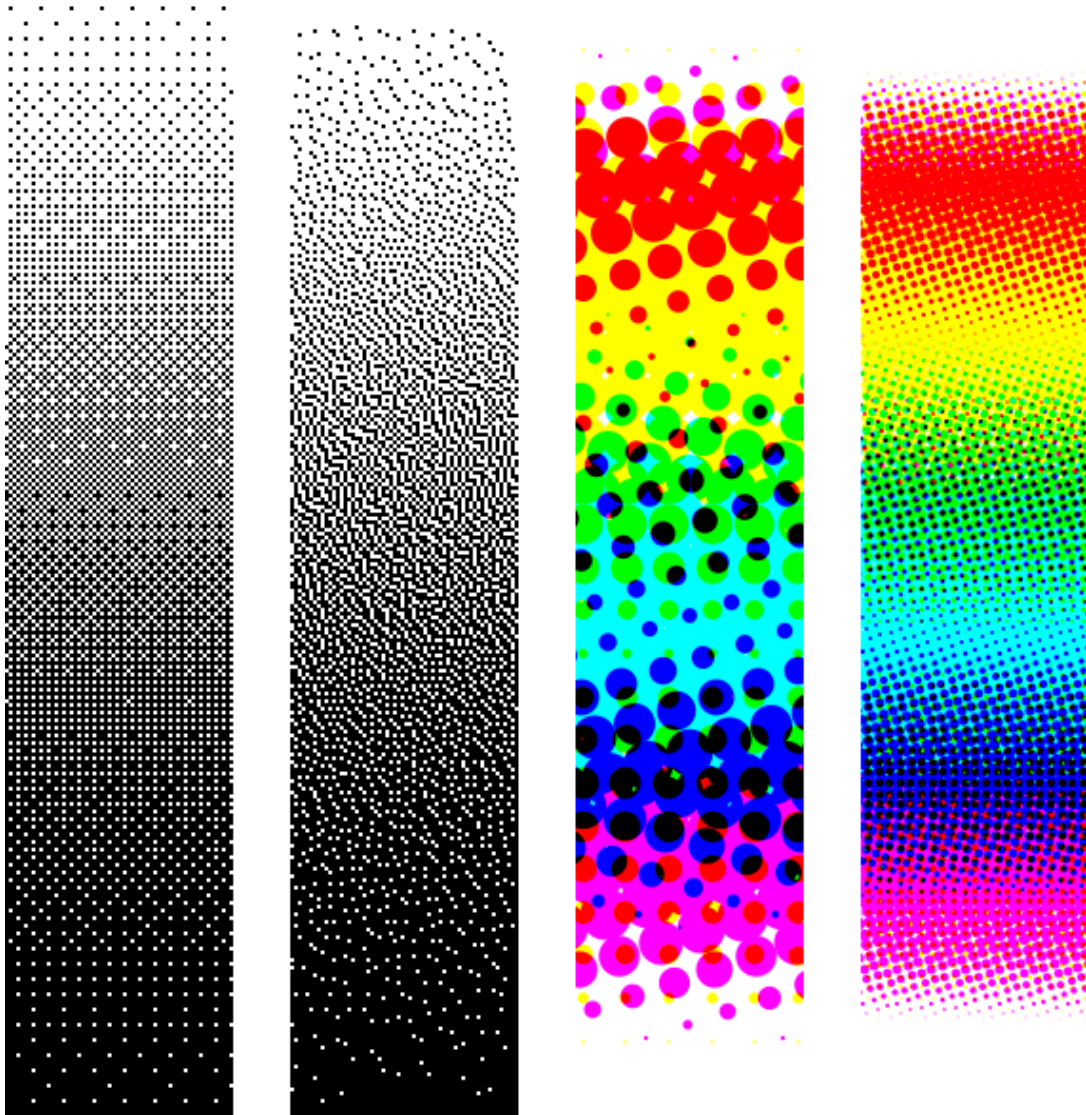
5.3.6 Blending

Reference material [here](#).

- **Blending** must be enabled for the material
- Blending is the process of determining a pixel's color by blending the color already in the framebuffer with the color passed from the fragment shader, taking into account the pixel's alpha value
- This is done using a series of **blending factors** (coefficients in the blending equation)
 - Let the *source blending factors* be given by (S_r, S_g, S_b, S_a)
 - Let the *destination blending factors* be given by (D_r, D_g, D_b, D_a)
 - Let the source pixel color be given by (R_s, G_s, B_s, A_s)
 - Let the destination's current pixel color be given by (R_d, G_d, B_d, A_d)
 - Then, the final blended color is given by:
 $(S_r R_s + D_r R_d, S_g G_s + D_g G_d, S_b B_s + D_b B_d, S_a A_s + D_a A_d)$
- The blending factors are controlled by `material.blending` and `material.blendEquation` which is set equal to one of the equations listed above.
- Addition is not the only operation that can be done to determine the blended color
 - Use the blend equation to change the way the pixels are blended (without the GLs)

5.3.7 Dithering

- **Dithering** is the process of improving color resolution at the expense of spatial resolution
 - Similar to “half-toning”, which is what newspapers do to show grey in images even though they have only two possible print colors - black and white
 - Spatial resolution is lost because neighboring pixels that should be same/similar colors are used to present a different color to achieve the effect
- Turned on or off with `{dither:true}` - no other settings are accessible or adjustable (without the GLs)



5.4 Fragment Shaders

- Another good example of using the Fragment Shader (with some additional framework for the render loop) is on Classroom
- Fragment shaders are the last step before the render to the screen takes place.
- Another built-in variable, `gl_FragCoord`, represents the pixel coordinate associated with the current fragment
 - This is what is output to the renderer, which will draw the associated pixel with the color
 - The “associated pixel” is the pixel that lives at the (x,y) coordinate of the fragment (described below)
 - `gl_FragCoord` contains window-space pixel coordinates $(x, y, z, 1/w)$. The (x, y) values are actual screen pixel coordinates (not clipping space coordinates), z represents depth in $[0, 1]$, and w holds $1/w_{clip}$.
 - Note that multiple fragments can have the same (x,y) of `gl_FragCoord` and may even have the same z-value, but that results in **z-fighting**
 - These coordinates are “window-relative”, or in pixel-space in other words
- A lot of what you can do in fragment shaders stems from access to Framebuffer Objects, which is not yet available in WebGL.
- Some really cool effects from fragment shaders:
<https://thebookofshaders.com/>
- Even more fan-contributed shaders:
<https://www.shadertoy.com/>

5.5 Practice

- Create two spheres that overlap partially from each other (make them red and green in that relative Z order).
- Use the fragment operations described in this section to swap the order of the spheres so that the green sphere is in the front and the red is in the back.
- Use the fragment operations to trim the screen so that only the middle part of it is drawn and the rest is black (so that it would look like a black frame or border around a small rectangle)
- Now turn on **blending**
 1. After drawing the spheres the first time, blend them so that the last sphere drawn uses the “max” color channel of all the spheres.
 2. What happens when this spheres is in front of the others?
- Change the colors of the red sphere to be dependent on the vertex positions (check out the example at https://2pha.com/demos/threejs/shaders/vector_position_color.html)
- Use a fragment shader to *invert* the colors
- Submit this practice to Canvas for 2 points.

5.6 Texture Mapping

- Because the surface of objects is often significantly more complex than we can capture by simply identifying the color of vertices/triangles, we use *texture mapping*
- It is similar to applying/glueing an image to a surface
- Usable in any shader, but most often used in the fragment shader
- Textures are made up of *texels*
- They are treated as simple tables with values that can be looked up, which makes them extremely powerful
- Using a texture in OpenGL/WebGL requires the following setup:
 - Create a *texture object* and load texel data into it
 - Include *texture coordinates* with your vertices
 - Associate a *texture sampler* with each texture map you intend to use in your shader
 - Retrieve the texel values through the texture sample from your shader
- In Three.js, textures are mapped to **materials**
 - They can still be accessed in the shader program, but it is not necessary
 - Texels are still important to understand, however, since access to them allows shader-level manipulation of the individual components of the texture.

5.7 Image Textures in Three.js

- Creating and using textures in Three.js requires the use of a `TextureLoader`

```
var loader = new THREE.TextureLoader();

var material = new THREE.MeshBasicMaterial({
  map: loader.load('https://threejsfundamentals.org/threejs/
resources/images/wall.jpg'),
});
```

- **Note:** The url can be local to your project directory, relative to the HTML file
- As an example: <https://threejsfundamentals.org/threejs/threejs-textured-cube.html>

```

import * as THREE from 'three';

function main() {
  const canvas = document.querySelector('#c');
  const renderer = new THREE.WebGLRenderer({canvas});

  const fov = 75;
  const aspect = 2; // the canvas default
  const near = 0.1;
  const far = 5;
  const camera =
    new THREE.PerspectiveCamera(fov, aspect, near, far);
  camera.position.z = 2;

  const scene = new THREE.Scene();

  const boxWidth = 1;
  const boxHeight = 1;
  const boxDepth = 1;
  const geometry =
    new THREE.BoxGeometry(boxWidth, boxHeight, boxDepth);

  const cubes = []; // an array we use to rotate the cubes

  // Here, we create the material and load it
  const loader = new THREE.TextureLoader();
  const material = new THREE.MeshBasicMaterial({
    // Use the map attribute to hold the texture
    map: loader.load('https://threejsfundamentals.org/threejs/
resources/images/wall.jpg'),
  });
  const cube = new THREE.Mesh(geometry, material);
  scene.add(cube);
  cubes.push(cube); // add to our list of cubes to rotate

  function resizeRendererToDisplaySize(renderer) {
    const canvas = renderer.domElement;
    const width = canvas.clientWidth;
    const height = canvas.clientHeight;
    const needResize =
      canvas.width !== width || canvas.height !== height;
    if (needResize) {
      renderer.setSize(width, height, false);
    }
    return needResize;
  }

  function render(time) {
    time *= 0.001;

    if (resizeRendererToDisplaySize(renderer)) {
      const canvas = renderer.domElement;
      camera.aspect = canvas.clientWidth / canvas.clientHeight;
      camera.updateProjectionMatrix();
    }
  }

```

```

cubes.forEach((cube, ndx) => {
  const speed = .2 + ndx * .1;
  const rot = time * speed;
  cube.rotation.x = rot;
  cube.rotation.y = rot;
});

renderer.render(scene, camera);

requestAnimationFrame(render);
}

requestAnimationFrame(render);
}

main();

```

5.7.1 Multiple Textures

- If you want to add multiple textures to the cube, all coming from different files, you can

```

const loader = new THREE.TextureLoader();
const materials = [
  new THREE.MeshBasicMaterial({map: loader.load('f-1.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-2.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-3.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-4.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-5.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-6.jpg')}),
];
const cube = new THREE.Mesh(geometry, materials);

```

- Not all geometry types supports multiple materials.
 - BoxGeometry and BoxBufferGeometry can use 6 materials one for each face
 - ConeGeometry and ConeBufferGeometry can use 2 materials, one for the bottom and one for the cone
 - CylinderGeometry and CylinderBufferGeometry can use 3 materials, bottom, top, and side
 - For other cases you will need to build or load custom geometry and/or modify texture coordinates

5.7.2 Asynchronous Texture Loading

- To wait for a texture to load the load method of the texture loader takes a callback that will be called when the texture has finished loading

```

const loader = new THREE.TextureLoader();
loader.load('resources/images/wall.jpg', (texture) => {
  const material = new THREE.MeshBasicMaterial({
    map: texture,
  });
  const cube = new THREE.Mesh(geometry, material);
  scene.add(cube);
  cubes.push(cube); // add to our list of cubes to rotate
});

```

- To wait until all textures have loaded you can use a `LoadingManager`, create one and pass it to the `TextureLoader` then set its `onLoad` property to a callback

```
const loadManager = new THREE.LoadingManager();
const loader = new THREE.TextureLoader(loadManager);

const materials = [
  new THREE.MeshBasicMaterial({map: loader.load('f-1.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-2.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-3.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-4.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-5.jpg')}),
  new THREE.MeshBasicMaterial({map: loader.load('f-6.jpg')}),
];

loadManager.onLoad = () => {
  const cube = new THREE.Mesh(geometry, materials);
  scene.add(cube);
};
```

5.8 Other Aspects of Textures

- Textures are *large* in terms of the memory they require, and your web sites will have limited memory and their loading speed will often depend on that memory
- Therefore, it's important to understand a few concepts
 - The two primary image file types are .png (lossless compression, larger files, slower to load) and .jpg (lossy compression)
 - Texels live in *texture coordinates*, a system where each texel has a *u* and a *v* coordinate, and each location on the model that a texel gets mapped to has an *s* and a *t* coordinate
- If the surface you are applying the texture to is too large or too small for the texture, what happens?
 - First, understand what a *mipmap* is - copies of the texture, each one half as wide and half as tall as the previous mip where the pixels have been blended to make the next smaller mip



- When the surface is smaller than original image, one of the *mips* is chosen
 - When the surface is larger, then the system supersamples the texels (spreads the texels out and averages them to determine the colors of the texels in the gaps)
- You can tell three.js what to do in these situations (see link below for examples)

5.8.1 Working with Textures on Models

- Texture objects have settings for repeating, offsetting, and rotating the texture
- By default, textures don't repeat
 - The texture's `wrapS` (for horizontal) and `wrapT` (for vertical) properties
 - `someTexture.wrapT = THREE.RepeatWrapping;` for example (for additional examples, see link below)
- If any texture settings are changed, you should set `texture.needsUpdate = true` to update them in `three.js`

For information about textures at

<https://threejsfundamentals.org/threejs/lessons/threejs-textures.html>

For additional information about textures,

<https://discoverthreejs.com/book/first-steps/textures-intro/>

5.8.2 Tex Coordinates (using Texels) in Three.js

Here is an example of using U.V. tex coordinates in Three.js with buffer geometry:

```
// This code adapted from:
// https://stackoverflow.com/questions/19504337/three-js-
//   buffergeometry-with-texture-coordinates
var scene = new THREE.Scene();
var camera = new THREE.PerspectiveCamera( 75,
    window.innerWidth / window.innerHeight, 0.1, 1000 );
var renderer = new THREE.WebGLRenderer();
renderer.setSize( window.innerWidth, window.innerHeight );
document.body.appendChild( renderer.domElement );

var quad_vertices =
[
    -30.0,  30.0,  0.0,
     30.0,  30.0,  0.0,
     30.0, -30.0,  0.0,
    -30.0, -30.0,  0.0
];
var quad_uvs =
[
    0.0, 1.0,
    1.0, 1.0,
    1.0, 0.0,
    0.0, 0.0
];
var quad_indices =
[
    0, 2, 1, 0, 3, 2
];

var geometry = new THREE.BufferGeometry();
var vertices = new Float32Array( quad_vertices );
var uvs = new Float32Array( quad_uvs );
var indices = new Uint32Array( quad_indices );

// Replaced deprecated geometry.addAttribute with geometry.
//   setAttribute
geometry.setAttribute( 'position', new THREE.BufferAttribute( vertices
    , 3 ) );
geometry.setAttribute( 'uv', new THREE.BufferAttribute( uvs, 2 ) );
geometry.setIndex( new THREE.BufferAttribute( indices, 1 ) );

// Load the texture asynchronously
var textureLoader = new THREE.TextureLoader();
textureLoader.load('texture.jpg', function (texture) {
    var material = new THREE.MeshBasicMaterial({ map: texture });
    var mesh = new THREE.Mesh( geometry, material );

    mesh.position.z = -100;
    scene.add(mesh);

    renderer.render(scene, camera);
}, undefined, function (err) {
    console.error('texture not loaded', err);
});
```

5.9 Practice

1. Create a new practice solution.
2. Draw a single quad on the screen.
3. Texture the quad.
 - (a) Find at least three png images online or on your computer
 - (b) Be careful with the texture coordinates and mapping them to your vertex coordinates
 - (c) Texture your quad with any of the images.
 - (d) When the user presses the 't' key, your program should cycle between the textures.